czii-cryo-et-object-identification

1st place solution [Object Detection Part] +Train Code +Inference +Models released

Rank	1
Team	Daddies
Author	Eugene Khvedchenya
Topic ID	561440
Version	Original

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561440
Retrieved with Kaggle CLI on 2026-10-21.

```
Throughout the competition there were numerous missile strikes, bombings, and other acts of war that
have taken the lives of many innocent people in Ukraine.
Rockets from russia hit within few kilometers from my home in Odesa. Each day Kaggle users from
Ukraine facing the chance of not waking up. Just keep in this mind while you read this solution
writeup.

I would like to thank the Armed Forces of Ukraine, the Security Service of Ukraine, Defence
Intelligence of Ukraine, and the State Emergency Service of Ukraine for providing safety and security
to participate in this great competition, complete this work, and help science, technology, and
business not to stop but to move forward.
```

TLDR: The solution in an ensemble of segmentation (3D Unets with ResNet & B3 encoders) and object detection models (SegResNet and DynUnet backbones) from Monai.
We accelerate model inference with NVidia TensorRT to achieve 200% speedup compared to eager PyTorch runtime and leverage two T4 GPUs to run predictions in parallel.

Kudos to my teammate @christofhenkel for his great work on ensembling our models into a final submission. Alone, we were able to get into Top-5, but when merged jumped immediately to the Top-1.
This writeup covers my part of the solution. Please check out Christof's writeup for segmentation approach and ensembling technique.

Update Training code, models and inference code for OD part released. Check it out here:

- Training code https://github.com/BloodAxe/Kaggle-2024-CryoET
- Pre-trained models dataset https://www.kaggle.com/datasets/bloodaxe/cryoet-detection-models (Mirror on GitHub: https://github.com/BloodAxe/Kaggle-2024-CryoET/releases/tag/1.0)
- [Notebook] Conversion from ONNX to TensorRT engine https://www.kaggle.com/code/bloodaxe/convert-onnx-ensemble-to-tensorrt
- [Notebook] Inference notebook (OD only) https://www.kaggle.com/code/bloodaxe/v4-inference-notebook/notebook
- [Dataset] ONNX ensemble https://www.kaggle.com/datasets/bloodaxe/cryoet-onnx-models/

Introduction

My baseline was a segmentation (heatmap) 3D Unet built from the SegResNet backbone. As a supervised objective target, I built a five-class heatmap with Gaussian peaks for present ground-truth targets in volume. To train a model I used reduced focal loss from CenterNet paper and NMS postprocessing approach from the same paper.
Although this approach immediately gave ~0.740 LB score (silver zone at the moment of submitting), I ditched it as I anticipated many participants would be using the segmentation approach. It is the easiest to implement, however it doesn't necessarily suit the best for the point detection task (A lot of model's capacity dedicated to predict the curvature of Gaussian at the peak and not on the particles discrimination). Second reason is model diversity. So if I decide to team up in the future, there will be less diversity in the final model ensemble.

Modeling approach

My final approach uses SegResNet and DynUnet backbones from Monai in combination with a custom point detection head. Based on my experience creating YOLO-based object detection models, I implemented an

anchor-free point detection model. Likewise to box detection, model predict class probabilities map and offsets to the center object. Unlike box detection, however there is no concept of bounding box and hence to use IoU between two points I used well-known object keypoint similarity measure `exp(-mse(x,y)/ (2 * radius ^ 2))` as proxy for IoU metric.

To train such model I implemented a custom loss function that mimics PP-Yolo loss function with a few modifications:

- Assignment of ground-truth labels to predicted centers based on point-point IoU. Point-Point IoU metric.
- Taking Top-K predictions for each GT label for training
- Computing varifocal loss for class map prediction and IoU-based loss for distance regression

My point detection approach worked well in terms of accuracy and training speed. On 4x3090 it took a mere two hours to train a single fold. The first submission of a single-fold detection model scored 0.752 on the LB. Once the modeling approach became clear, the next effort was to make it fast.

In this competition, we need to process 500 scans within a 12-hour time limit. This translates into ~1m30s per single study. In my models, I predict class-map `[B,C,D/2,H/2,W/2]` and offsets map `[B,3,D/2,H/2,W/2]` in stride 2 along depth, height, and width.

For the object detection approach, there is no need to predict a full-resolution feature maps. Predicting smaller feature maps has a massive impact on model throughput. I've got almost 50% speed increase when changing to stride 2 output from stride 1 (full resolution).

I ablate on stride 1, stride 2, stride 4, and stride 2 & 4 outputs and found that:

- Stride 1 gives a slightly, slightly better (0.002) LB score than stride 2
- Stride 4 also worked ok, but stride 2 was better in the F-beta score.
- Using stride 2 and 4 didn't bring any improvements over using only stride 2. An interesting observation when using the two-heads method: Large particles tend to migrate to stride 4 while small classes were present on the class-map with stride 2.

In all my models I use stride 2 prediction maps.

Training

I used 5-fold cross-validation scheme for all models. Each fold used 2 studies for validation and 5 for training.

Training epoch used fixed number of random crops per study and fixed number of random crops around each particle instance. For data augmentations I used random rotations along Z-axis (+- 180 degrees), slight rotations along X and Y axis (+-10), subtle scale jitter (+-5%), and random flips along X, Y, Z axes.
I experimented with erasing particles from the input volume, random copy-pasting particles from one study to another, doing mixup on the particle instances, but these augmentation techniques didn't bring any improvements in the final model.

Validation epoch used sliding window approach with the same window size and overlap as in the inference kernel. During validation individual tiles accumulated to final classmap and offsets map and F-beta score was computed on the final maps. After each epoch, I computed per-class thresholds that maximizes F-beta score on the validation set. I saved top-5 models for training experiment which I later averaged which almost always increased the F-beta score.

Training used 96x128x128px while validation was performed on 192x128x128 volumes.

Tiling

I used a sliding window approach to tile the input volume. The window size was 192x128x128px with 1x9x9 tiles configuration for input volume of 184x630x630. Individual tiles were aggregated to final classmap and offsets map using a weighted averaging approach where pixels on the border of the tile had lower weights than pixels in the center of the tile. For sliding average aggregation I used my own implementation.

Postprocessing

Having an aggregated classmap of [C, D, H, W] shape and offsets map of [3, D, H, W] shape, I used the following postprocessing steps:

- Centernet-like NMS to remove duplicate detections: `x = x * (x == maxpool3d(x, kernel_size=3, stride=1, padding=1))`
- Take Top-K predictions for each class (I used 16K but lower numbers should also work fine I guess)
- Filter out predictions by confidence threshold (Each class has it's own confidence threshold)
- Apply greedy NMS using pairwise IoU distance (Similar to how Box NMS works) for detections of a single class.
- Scale pixels to Angstroms

Since my models were directly predicting offset to the center of the particle, this approach is not sensitive to systematic shift in point coordinates (Some argue it's 0.5px, some claims it to be 1px).

Object Detection Ensemble

My final object ensemble is 5 models (folds) of OD SegResNet and 5 models of OD DynUnet (OD stands for Object Detection to differentiate them from segmentation-based models).
A 10 models in total. I used local F-beta metric to select final checkpoints that would make into ensemble.

SegResNet

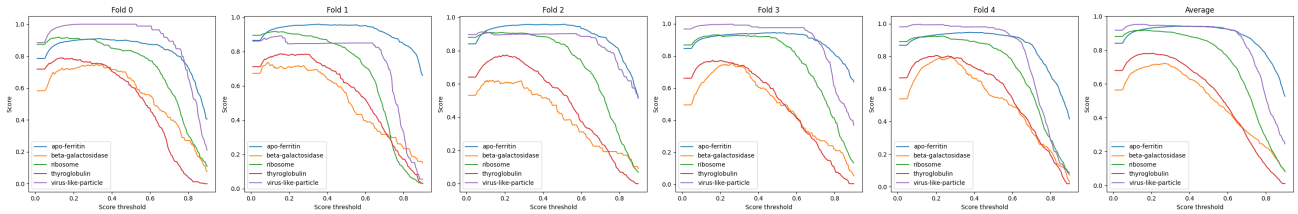
fold	score	AFRT	BGT	RBSM	TRGLB	VLP
0	0.8457	0.265	0.29	0.195	0.15	0.55
1	0.8366	0.345	0.11	0.185	0.135	0.305
2	0.8046	0.355	0.28	0.395	0.34	0.185
3	0.8398	0.23	0.345	0.405	0.36	0.255
4	0.8437	0.165	0.35	0.245	0.235	0.27

DynUnet

fold	score	AFRT	BGT	RBSM	TRGLB	VLP
0	0.8272	0.145	0.33	0.215	0.235	0.405
1	0.8337	0.39	0.185	0.215	0.21	0.235
2	0.7971	0.425	0.16	0.55	0.275	0.65
3	0.8418	0.385	0.345	0.345	0.305	0.21
4	0.842	0.23	0.34	0.255	0.375	0.14

As you can see on F-4 scores from individual folds, there is a serious gap between CV and LB scores. Me explanation of this phenomenon is big difference in number of studies in validation set that we use locally and LB. I found that extending validation set by using flipped and 90-degree rotated studies lowers the CV score and also makes F-beta curve mode smooth, which allow finding a more accurate and less noisy threshold.

To figure out thresholds of the final ensemble I compute OOF F-beta curves for each class and select thresholds that maximize average F-beta score:



Accelerating inference

Final submission uses TensorRT for inference.

Initially, I used `torch.jit` which was enough for start, but as we teamed up with @christofhenkel a more efficient approach was needed.
I tried using `onnxruntime` with `CUDAExecutionProvider` which gave me nearly the same throughput as `torch.jit`.
Finally, by enabling `TensorRTExecutionProvider` that leverages all the power of TensorRT I was able to achieve desired speedup of 200% compared to `torch.jit`

The proces of going from individual checkpoints to TensorRT engine is multi-stage:

1. [Offline] Convert individual checkpoints into a single ONNX model containing all models and averaging their predictions
2. [Kaggle] Convert ONNX model into TensorRT engine (And save it to disk). This step happens on Kaggle, using T4 GPU (a target GPU we use for inference) and it saved us 10 minutes of submission time for creating TensorRT engine.
3. [Kaggle] Actual inference notebook. I split all test data in two chunks and use 2xT4 GPUs to process them in parallel. Then two submission shards were concatenated to form the final submission.

To combine results of my ensemble with ensemble of @christofhenkel we simply used weighted average of the predictions on the classmap level followed
by postprocessing described above. Please check second part of the solution here.

Things that didn't quite work (for me)

- Mixup, Copy-Paste, Random-Erasing (CV higher, LB lower)
- 2.5-D models
- 3D version of HRNet and ConvNext
- Gaussian noise, anisotropic scale jitter
- Knowledge distillation

References

1. Training code https://github.com/BloodAxe/Kaggle-2024-CryoET
2. Pre-trained models dataset https://www.kaggle.com/datasets/bloodaxe/cryoet-detection-models (Mirror on GitHub: https://github.com/BloodAxe/Kaggle-2024-CryoET/releases/tag/1.0)
3. [Notebook] Conversion from ONNX to TensorRT engine https://www.kaggle.com/code/bloodaxe/convert-onnx-ensemble-to-tensorrt
4. [Notebook] Inference notebook (OD only) https://www.kaggle.com/code/bloodaxe/v4-inference-notebook/notebook
5. [Dataset] ONNX ensemble https://www.kaggle.com/datasets/bloodaxe/cryoet-onnx-models/

Supplemental Q&A

> **Yassine Alouini · 2025-02-11T11:26:32.770000**
>
> Thanks for the write-up.
> You mention that MixUp didn't work for the object detection part but it seems that it has worked for the segmentation part.
> Do you have any ideas why?

> **Eugene Khvedchenya · 2025-02-13T09:37:26.290000**
>
> It's a good question to ask. Frankly - I don't know. I imagine if we have at least 100 studies in train, the CV estimate would be less noisy and we could obtain more reliable signals on what work and what doesn't.
>
> The metric is very sensitive to confidence thresholds you choose, so LB validation was the only viable approach to tune thresholds, given the size of studies in public part of the LB. I don't quite like overfitting to public, but in this comp it was the only viable choice. Under these conditions, it is really hard to say confidently that one approach works or doesn't since you may be just in poor thresholds area and moving one threshold to +0.05 and another -0.05 would give you SOTA. I tried playing with thresholds a bit for mixup models but didn't spent more than 5 subs on finding best combination of thresholds.